TOPIC 06
INPUT AND OUTPUT

Fundamentals of Computing
(FSPK0022)

Input and Output in C++

- C++ uses a convenient abstraction called *streams* to perform input and output operations in sequential media such as the screen, the keyboard or a file.
- C++ iostream - `#include <iostream>`

  i – input

  cin : standard **input stream**

  can read the input from keyboard

  o – output

  cout : standard **output stream**

  can display output to the screen

Input using `cin`

The `cin` Object

- Standard input object
- Like **`cout`**, requires **`iostream`** file
- Used to read input from **keyboard**
- Information retrieved from **`cin`** with **`>>`**
- Input is stored in one or more variables

Program 3-1

```
1  // This program asks the user to enter the length and width of
2  // a rectangle. It calculates the rectangle's area and displays
3  // the value on the screen.
4  #include <iostream>
5  using namespace std;
6
7  int main()
8  {
9     int length, width, area;
10
11    cout << "This program calculates the area of a ";
12    cout << "rectangle.\n";
13    cout << "What is the length of the rectangle? ";
14    cin >> length;
15    cout << "What is the width of the rectangle? ";
16    cin >> width;
17    area = length * width;
18    cout << "The area of the rectangle is " << area << ".\n";
19    return 0;
20 }
```

Program Output with Example Input Shown in Bold

```
This program calculates the area of a rectangle.
What is the length of the rectangle? 10 [Enter]
What is the width of the rectangle? 20 [Enter]
The area of the rectangle is 200.
```

The `cin` Object

- **`cin`** converts data to the type that matches the variable:

```
int height; //variable declaration
cout << "How tall is the room? ";
cin >> height;
```

The `cin` Object

- Can be used to input more than one value:

```
cin >> height >> width;
```

- Multiple values from keyboard must be separated by spaces

- Order is important: first value entered goes to first variable, etc.

Displaying a Prompt

- A prompt is a message that instructs the user to enter data.
- You should always use **`cout`** to display a prompt before each **`cin`** statement.

```
cout << "How high is the room? ";
cin >> height;
```

Program 3-2

```
1  // This program asks the user to enter the length and width of
2  // a rectangle. It calculates the rectangle's area and displays
3  // the value on the screen.
4  #include <iostream>
5  using namespace std;
6
7  int main()
8  {
9     int length, width, area;
10
11    cout << "This program calculates the area of a ";
12    cout << "rectangle.\n";
13    cout << "Enter the length and width of the rectangle ";
14    cout << "separated by a space.\n";
15    cin >> length >> width;
16    area = length * width;
17    cout << "The area of the rectangle is " << area << endl;
18    return 0;
19 }
```

Program Output with Example Input Shown in Bold

```
This program calculates the area of a rectangle.
Enter the length and width of the rectangle separated by a space.
10 20 [Enter]
The area of the rectangle is 200
```

In-Class Exercise

Write the code to solve the problem below:

Sample of output:

Enter an integer: **7**
Enter a decimal number : **2.25**
Enter two odd numbers : **3 19**

Output using `cout`

The `cout` Object

- Displays information on computer screen
- Use **<<** to send information to **`cout`**

```
cout << "Hello, there!";
```

- Can use **<<** to send multiple items to **`cout`**

```
cout << "Hello, " << "there!";
```

Or

```
cout << "Hello, ";
cout << "there!";
```

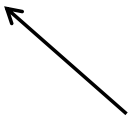
Starting a New Line

- To get multiple lines of output on screen
  - Use **endl**

```
cout << "Value, 1!" << endl;
```

  - Use **\n** in an output string

```
cout << "Value, 1!\n";
```



Notice that the \n is INSIDE the string.

In-Class Exercise

- Rearrange the following program statements in the correct order.

```
int main()
}
return 0;
float p=9.81;
#include <iostream>
cout << p;
cout << "Display the air pressure :";
{
using namespace std;
```

- What is the output of the program when it is properly arranged?

Formatting Output

Introduction to Output Formatting

- Can **control how output displays** for numeric and string data:
  - size
  - position
  - number of digits
- Done through the use of manipulators, special variables or objects that are placed on the output stream.
- Most of the standard manipulators are found in **<iostream>**, some requires <**iomanip**> header file.

Stream Manipulators

Stream Manipulator	Description
setw(*n*)	Establishes a print field on *n* spaces.
fixed	Displays floating-point numbers in fixed point notation.
setprecision(*n*)	Sets the number of significant digits or floating-points.

Formatting Output: `setw()`

- Used to ouput the value of an expression in a **specific number of columns**
- **setw(x)** - outputs the value of the next expression in x columns
- The output is **right-justified**
  - *Example:* if you specify the number of columns to be 8 and the output requires only 4 columns, then the first four columns are left blank
- If the number of **columns specified is less than** the number of **columns required** by the output, the output **automatically expands** to the required number of columns

Example 1: `setw()`

Program 3-16

```
1  // This program displays three rows of numbers.
2  #include <iostream>
3  #include <iomanip>      // Required for setw
4  using namespace std;
5
6  int main()
7  {
8     int num1 = 2897, num2 = 5,    num3 = 837,
9         num4 = 34,   num5 = 7,    num6 = 1623,
10        num7 = 390,  num8 = 3456, num9 = 12;
11
12    // Display the first row of numbers
13    cout << setw(6) << num1 << setw(6)
14         << num2 << setw(6) << num3 << endl;
15
16    // Display the second row of numbers
17    cout << setw(6) << num4 << setw(6)
18         << num5 << setw(6) << num6 << endl;
19
```

Program 3-16 *(continued)*

```
20    // Display the third row of numbers
21    cout << setw(6) << num7 << setw(6)
22         << num8 << setw(6) << num9 << endl;
23    return 0;
24 }
```

Program Output

```
  2897     5   837
    34     7  1623
   390  3456    12
```

Example 2: `setw()`

```
#include <iostream>
#include <iomanip>

using namespace std;

int main()
{
  cout << "*" << -17 << "*" << endl;
  cout << "*" << setw(6) << -17 << "*" << endl << endl;

  cout << "*" << "Hi there!" << "*" << endl;
  cout << "*" << setw(20) << "Hi there!" << "*" << endl;
  cout << "*" << setw(3) << "Hi there!" << "*" << endl;

  return 0;
}
```

Example 1: `fixed`

```
#include <iostream>
using namespace std;

int main()
{
    double x = 15.674;
    double y = 235.73;
    double z = 9525.9874;

    cout << fixed;
    cout << x << endl << y << endl << z << endl;

    return 0;
}
```

Example 2: `fixed`

```
#include <iostream>
using namespace std;

int main()
{
  float small = 3.14159265358979323846;
  float large = 6.0234567e17;
  float whole = 2.000000000;
  cout << "Some values in general format" << endl;
  cout << "small:  " << small << endl;
  cout << "large:  " << large << endl;
  cout << "whole:  " << whole << endl << endl;

  cout << fixed;
  cout << "The same values in fixed format" << endl;
  cout << "small:  " << small << endl;
  cout << "large:  " << large << endl;
  cout << "whole:  " << whole << endl << endl;
  return 0;
}
```

`setprecision()` Manipulator

- To **control** the **number of significant digits (or precision)** of the output, i.e., the total number of digits before and after the decimal point.
- However, when **used with fixed**, it specifies the **number of floating-points** (i.e., the number of digits after the decimal point).
- **Without fixed**, the `setprecision()` is **set to a lower value**, it will print floating-point value using **scientific notation**.
- **`setprecision(`*`n`*`)`** – ***n*** is the number of **significant digits** or the number of **floating-point** (if used with **`fixed`**).

Example 1: setprecision()

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    double x = 15.674;
    double y = 235.73;
    double z = 9525.9874;

    cout << setprecision(2);
    cout << x << endl << y << endl << z << endl;
    return 0;
}
```

Example 2: setprecision()

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
  double x = 156.74, y = 235.765, z = 9525.9874;

  cout << setprecision(5) << x << endl;
  cout << setprecision(3) << x << endl;
  cout << setprecision(2) << x << endl;
  cout << setprecision(1) << x << endl;

  cout << fixed << setprecision(2);
  cout << x << endl << y << endl << z << endl;

  return 0;
}
```

Formatted Input

Introduction to Input Formatting

- Can format field width for use with **cin**.
- Useful when reading string data to be stored in a character array:

```
const int SIZE = 10;
char firstName[SIZE];
cout << "Enter your name: ";
cin >> setw(SIZE) >> firstName;
```

- **cin** reads one less character than specified with the **setw()** manipulator.

Example: Input Formatting

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    const int SIZE = 10;
    char firstName[SIZE];

    cout << "Enter your name: ";
    cin >> setw(SIZE) >> firstName;
    cout << firstName << endl;

    return 0;
}
```

Example: Problem using `cin`

```
#include <iostream>
using namespace std;

int main()
{
    string name;
    cout << "Enter your name: ";
    cin >> name;
    cout << name << endl;
    return 0;
}
```

Input Formatting: `getline()`

- To read an entire line of input, use **`getline()`**.
- When reading string data to be stored in a **character array**, use **`getline()`** with two arguments:
  - Name of array to store string
  - Size of the array

Example: getline()

```
#include <iostream>
using namespace std;

int main()
{
    const int SIZE = 20;
    char firstName[SIZE];

    cout << "Enter your name: ";
    cin.getline (firstName, SIZE);
    cout << firstName << endl;

    return 0;
}
```

Input Formatting: `get()`

- To read a single character, use **`cin`**.

```
char ch;
cout << "Strike any key to continue";
cin >> ch;
```

Problem: will **skip over** blanks, tabs, <ENTER>

- Solution to read a single character, use **`get()`**

```
char ch;
cout << "Strike any key to continue";
cin.get(ch);
```

Advantage: Will **read the next character entered**, even **whitespace**.

Input Formatting: `ignore()`

- **Mixing `cin >>`** and **`cin.get()`** in the **same program** can cause input errors that are hard to detect.
- To **skip over unneeded characters** that are still in the keyboard buffer, use **`cin.ignore()`**:

```
//skip next char
cin.ignore();
//skip the next 10 char. @ until a '\n'
cin.ignore(10,'\n');
```

.

In-Class Exercise

- What will be displayed if the user enters the following input:

```
202
L
```

```
#include <iostream>
using namespace std;

int main()
{
  int id;
  char code;
  cout << "Enter an integer id: ";
  cin >> id;
  cout << "Enter a code: ";
  cin.get(code);
  cout << "Output\n" << id << "\t" << code;
  return 0;
}
```